



**Politecnico
di Torino**

Politecnico di Torino

Corso di Laurea

A.a. 2025/2026

Sessione di laurea Mese Anno

Titolo della tesi

Eventuale sottotitolo

Relatori:

Name Surname

Name Surname

Candidati:

Omar Ferro

Ringraziamenti

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Tristique senectus et netus et malesuada fames ac turpis. Pretium nibh ipsum consequat nisl vel pretium lectus quam. Urna molestie at elementum eu facilisis sed odio morbi quis. Sed egestas egestas fringilla phasellus faucibus scelerisque eleifend. At in tellus integer feugiat. Mauris rhoncus aenean vel elit scelerisque mauris pellentesque pulvinar. Commodore sed egestas egestas fringilla. Nunc lobortis mattis aliquam faucibus purus in massa. Facilisis magna etiam tempor orci eu lobortis elementum nibh. Elementum curabitur vitae nunc sed velit dignissim. Neque volutpat ac tincidunt vitae. Massa id neque aliquam vestibulum morbi blandit cursus risus at. Porta non pulvinar neque laoreet suspendisse interdum consectetur. Turpis in eu mi bibendum. Ut tristique et egestas quis ipsum suspendisse. Integer quis auctor elit sed vulputate mi sit amet. Viverra nam libero justo laoreet sit amet cursus.

Indice

Elenco delle figure	VI
1 Introduzione	1
1.1 Motivazioni	1
1.2 Obiettivi	2
1.3 Contributi principali	2
1.4 Struttura del documento	2
2 Contesto e requisiti	3
2.1 Use case	3
2.2 Requisiti funzionali e non funzionali	3
3 Fondamenti teorici e tecnologie	5
3.1 Linguaggio Go	5
3.2 Modello pipeline	5
3.3 Lock-free data structures e ring buffer	6
4 Architettura della libreria	7
4.1 Pipeline	7
4.2 Stage	7
4.3 Connector	8
4.4 Worker Pool	9
4.5 Ingress Stage	9
4.5.1 Ticker	10
4.5.2 UDP	11
4.5.3 TCP	13
4.6 Stage di processamento	17
4.7 Stage di egress	18
4.8 Observabilità con OpenTelemetry	18

5	Implementazione	19
5.1	Organizzazione codice	19
5.2	Interfacce principali	19
5.3	Dettagli ring buffer	20
5.4	Dettagli worker pool	20
5.5	Dettagli ROB e stimatore EMA	21
6	Benchmark e valutazione prestazioni	22
6.1	Setup sperimentale	22
6.2	Confronto con altri framework	22
6.3	Analisi vantaggi e limiti	23
6.4	Impatto OpenTelemetry	23
7	Applicazioni e sviluppi futuri	24
7.1	Telemetria in SquadraCorse	24
7.2	Miglioramenti futuri	24
8	Conclusioni	26
	Bibliografia	27

Elenco delle figure

Capitolo 1

Introduzione

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Porttitor eget dolor morbi non arcu risus quis varius. Libero id faucibus nisl tincidunt. Neque laoreet suspendisse interdum consectetur libero id faucibus nisl tincidunt. Scelerisque in dictum non consectetur a erat. Leo a diam sollicitudin tempor id eu. Sodales ut eu sem integer vitae justo eget magna fermentum. A cras semper auctor neque vitae. Cursus euismod quis viverra nibh cras pulvinar. Mi tempus imperdiet nulla malesuada pellentesque elit eget gravida cum. Dictum at tempor commodo ullamcorper a lacus vestibulum sed. Ultricies mi eget mauris pharetra. In pellentesque massa placerat dui ultricies lacus. Fringilla phasellus faucibus scelerisque eleifend donec pretium vulputate.

Aliquam sem fringilla ut morbi tincidunt augue interdum. Risus at ultrices mi tempus imperdiet nulla malesuada pellentesque. Semper quis lectus nulla at volutpat. Nullam non nisi est sit amet facilisis. Eget velit aliquet sagittis id consectetur purus ut faucibus pulvinar. Ultricies tristique nulla aliquet enim tortor at auctor urna nunc. Pharetra diam sit amet nisl suscipit adipiscing bibendum est ultricies. Amet facilisis magna etiam tempor. Nunc non blandit massa enim nec dui nunc mattis. At ultrices mi tempus imperdiet nulla malesuada pellentesque. Cursus metus aliquam eleifend mi in nulla posuere. In eu mi bibendum neque egestas congue quisque. Augue eget arcu dictum varius dui at consectetur.

1.1 Motivazioni

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Tristique senectus et netus et malesuada fames ac turpis. Pretium nibh ipsum consequat nisl vel pretium lectus quam. Urna molestie at elementum eu facilisis sed odio morbi quis. Sed egestas egestas fringilla

phasellus faucibus scelerisque eleifend. At in tellus integer feugiat. Mauris rhoncus aenean vel elit scelerisque mauris pellentesque pulvinar. Commodos egestas egestas fringilla. Nunc lobortis mattis aliquam faucibus purus in massa. Facilisis magna etiam tempor orci eu lobortis elementum nibh. Elementum curabitur vitae nunc sed velit dignissim. Neque volutpat ac tincidunt vitae. Massa id neque aliquam vestibulum morbi blandit cursus risus at. Porta non pulvinar neque laoreet suspendisse interdum consectetur. Turpis in eu mi bibendum. Ut tristique et egestas quis ipsum suspendisse. Integer quis auctor elit sed vulputate mi sit amet. Viverra nam libero justo laoreet sit amet cursus.

1.2 Obiettivi

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Tristique senectus et netus et malesuada fames ac turpis. Pretium nibh ipsum consequat nisl vel pretium lectus quam. Urna molestie at elementum eu facilisis sed odio morbi quis. Sed egestas egestas fringilla phasellus faucibus scelerisque eleifend. At in tellus integer feugiat. Mauris rhoncus aenean vel elit scelerisque mauris pellentesque pulvinar. Commodos egestas egestas fringilla. Nunc lobortis mattis aliquam faucibus purus in massa. Facilisis magna etiam tempor orci eu lobortis elementum nibh. Elementum curabitur vitae nunc sed velit dignissim. Neque volutpat ac tincidunt vitae. Massa id neque aliquam vestibulum morbi blandit cursus risus at. Porta non pulvinar neque laoreet suspendisse interdum consectetur. Turpis in eu mi bibendum. Ut tristique et egestas quis ipsum suspendisse. Integer quis auctor elit sed vulputate mi sit amet. Viverra nam libero justo laoreet sit amet cursus.

1.3 Contributi principali

1.4 Struttura del documento

Capitolo 2

Contesto e requisiti

2.1 Use case

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Tristique senectus et netus et malesuada fames ac turpis. Pretium nibh ipsum consequat nisl vel pretium lectus quam. Urna molestie at elementum eu facilisis sed odio morbi quis. Sed egestas egestas fringilla phasellus faucibus scelerisque eleifend. At in tellus integer feugiat. Mauris rhoncus aenean vel elit scelerisque mauris pellentesque pulvinar. Commodo sed egestas egestas fringilla. Nunc lobortis mattis aliquam faucibus purus in massa. Facilisis magna etiam tempor orci eu lobortis elementum nibh. Elementum curabitur vitae nunc sed velit dignissim. Neque volutpat ac tincidunt vitae. Massa id neque aliquam vestibulum morbi blandit cursus risus at. Porta non pulvinar neque laoreet suspendisse interdum consectetur. Turpis in eu mi bibendum. Ut tristique et egestas quis ipsum suspendisse. Integer quis auctor elit sed vulputate mi sit amet. Viverra nam libero justo laoreet sit amet cursus.

2.2 Requisiti funzionali e non funzionali

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Tristique senectus et netus et malesuada fames ac turpis. Pretium nibh ipsum consequat nisl vel pretium lectus quam. Urna molestie at elementum eu facilisis sed odio morbi quis. Sed egestas egestas fringilla phasellus faucibus scelerisque eleifend. At in tellus integer feugiat. Mauris rhoncus aenean vel elit scelerisque mauris pellentesque pulvinar. Commodo sed egestas egestas fringilla. Nunc lobortis mattis aliquam faucibus purus in massa. Facilisis magna etiam tempor orci eu lobortis elementum nibh. Elementum curabitur vitae nunc sed velit dignissim. Neque volutpat ac tincidunt vitae. Massa id neque

aliquam vestibulum morbi blandit cursus risus at. Porta non pulvinar neque laoreet suspendisse interdum consectetur. Turpis in eu mi bibendum. Ut tristique et egestas quis ipsum suspendisse. Integer quis auctor elit sed vulputate mi sit amet. Viverra nam libero justo laoreet sit amet cursus.

Capitolo 3

Fondamenti teorici e tecnologie

3.1 Linguaggio Go

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Tristique senectus et netus et malesuada fames ac turpis. Pretium nibh ipsum consequat nisl vel pretium lectus quam. Urna molestie at elementum eu facilisis sed odio morbi quis. Sed egestas egestas fringilla phasellus faucibus scelerisque eleifend. At in tellus integer feugiat. Mauris rhoncus aenean vel elit scelerisque mauris pellentesque pulvinar. Commodo sed egestas egestas fringilla. Nunc lobortis mattis aliquam faucibus purus in massa. Facilisis magna etiam tempor orci eu lobortis elementum nibh. Elementum curabitur vitae nunc sed velit dignissim. Neque volutpat ac tincidunt vitae. Massa id neque aliquam vestibulum morbi blandit cursus risus at. Porta non pulvinar neque laoreet suspendisse interdum consectetur. Turpis in eu mi bibendum. Ut tristique et egestas quis ipsum suspendisse. Integer quis auctor elit sed vulputate mi sit amet. Viverra nam libero justo laoreet sit amet cursus.

3.2 Modello pipeline

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Tristique senectus et netus et malesuada fames ac turpis. Pretium nibh ipsum consequat nisl vel pretium lectus quam. Urna molestie at elementum eu facilisis sed odio morbi quis. Sed egestas egestas fringilla phasellus faucibus scelerisque eleifend. At in tellus integer feugiat. Mauris rhoncus aenean vel elit scelerisque mauris pellentesque pulvinar. Commodo sed egestas

egestas fringilla. Nunc lobortis mattis aliquam faucibus purus in massa. Facilisis magna etiam tempor orci eu lobortis elementum nibh. Elementum curabitur vitae nunc sed velit dignissim. Neque volutpat ac tincidunt vitae. Massa id neque aliquam vestibulum morbi blandit cursus risus at. Porta non pulvinar neque laoreet suspendisse interdum consectetur. Turpis in eu mi bibendum. Ut tristique et egestas quis ipsum suspendisse. Integer quis auctor elit sed vulputate mi sit amet. Viverra nam libero justo laoreet sit amet cursus.

3.3 Lock-free data structures e ring buffer

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Tristique senectus et netus et malesuada fames ac turpis. Pretium nibh ipsum consequat nisl vel pretium lectus quam. Urna molestie at elementum eu facilisis sed odio morbi quis. Sed egestas egestas fringilla phasellus faucibus scelerisque eleifend. At in tellus integer feugiat. Mauris rhoncus aenean vel elit scelerisque mauris pellentesque pulvinar. Commoda sed egestas egestas fringilla. Nunc lobortis mattis aliquam faucibus purus in massa. Facilisis magna etiam tempor orci eu lobortis elementum nibh. Elementum curabitur vitae nunc sed velit dignissim. Neque volutpat ac tincidunt vitae. Massa id neque aliquam vestibulum morbi blandit cursus risus at. Porta non pulvinar neque laoreet suspendisse interdum consectetur. Turpis in eu mi bibendum. Ut tristique et egestas quis ipsum suspendisse. Integer quis auctor elit sed vulputate mi sit amet. Viverra nam libero justo laoreet sit amet cursus.

Capitolo 4

Architettura della libreria

4.1 Pipeline

Una *pipeline*, in informatica, è una sequenza di stadi (*stage*) attraverso cui i dati fluiscono in modo ordinato. Ciascuno *stage* riceve un input, lo elabora e produce un output che diventa l'input dello *stage* successivo. Ogni *stage* ha una responsabilità ben delimitata, favorendo così modularità, riuso e facilità di ragionamento sull'intero sistema.

In contesti concorrenti, ogni *stage* può operare in parallelo sugli elementi che riceve. Ciò significa che, mentre lo *stage* 1 elabora un nuovo messaggio, lo *stage* 2 può già processare quello precedente, incrementando così il *throughput* complessivo della pipeline.

All'interno della libreria **goccia**, il concetto di *pipeline* è incarnato dalla struct **Pipeline**, che funge da orchestratore degli *stage* e rappresenta il punto di ingresso per l'utilizzo della libreria stessa. Questa struct definisce i tre metodi principali che ogni *stage* deve implementare: **Init**, **Run** e **Close**, corrispondenti alle diverse fasi del ciclo di vita della pipeline (si veda la sezione successiva per ulteriori dettagli). Inoltre, il metodo **AddStage** consente di aggiungere nuovi *stage* fintanto che la pipeline non è in esecuzione.

<disegno pipeline>

4.2 Stage

Uno *stage* di una pipeline rappresenta un'unità modulare di elaborazione che riceve un input da *stage* precedenti, oppure da una sorgente esterna, lo elabora e produce un output per gli *stage* successivi. In tal modo, gli *stage* formano una catena di trasformazione dei dati scalabile e facilmente componibile. Dal punto di vista concettuale, uno *stage* costituisce un limite logico e sequenziale all'interno della

pipeline, con un flusso dati che attraversa le fasi di input, elaborazione e output, in maniera simile a quanto avviene nei processi CI/CD (build, test, deploy).

Nella libreria **goccia**, gli *stage* devono aderire all'interfaccia **Stage**, la quale definisce i metodi necessari per rendere una struct di Go compatibile con la pipeline principale. Tali metodi coincidono con le tre fasi del ciclo di vita: **Init**, per l'inizializzazione delle risorse; **Run**, per l'elaborazione dei dati in input; e **Close**, per il rilascio delle risorse e la chiusura dello *stage*.

La libreria categorizza gli *stage* in tre gruppi principali — **Ingress**, **Processor** ed **Egress** — tutti aderenti all'interfaccia generica **Stage**. Un aspetto rilevante riguarda la modalità di esecuzione: il metodo **Run** viene invocato all'interno di una goroutine dedicata per ciascuno *stage*. Di conseguenza, una pipeline composta da cinque *stage* disporrà di almeno cinque goroutine attive, oltre a quella principale di esecuzione nel main.

<disegno lifecycle>

4.3 Connector

Affinché una *pipeline* possa effettivamente operare, una volta creati gli *stage*, è necessario metterli in comunicazione tramite un componente capace di trasferire i dati da uno *stage* al successivo. Tale componente deve essere *thread safe*, poiché ogni *stage* viene eseguito su una goroutine distinta, e deve disporre di un meccanismo di buffer per mitigare gli effetti della *backpressure*, fenomeno che si manifesta quando uno *stage* a valle elabora i dati più lentamente del precedente, causando un rallentamento a catena.

Il linguaggio Go fornisce, a tal fine, i channel (anche bufferizzati) come tipo primitivo per la comunicazione tra goroutine. Tuttavia, la libreria **goccia** adotta un approccio differente, basato su un *ring buffer* lock-free, con l'obiettivo di massimizzare il *throughput*. In particolare, si è optato per un'implementazione *Single Producer Single Consumer* (SPSC), poiché la relazione tra gli *stage* della pipeline è di tipo 1:1. La versione *Multiple Producer Multiple Consumer* (MPMC) del *ring buffer* viene invece impiegata nel contesto del *worker pool* (descritto successivamente).

<tabella channel vs ring buffer>

Per mantenere la libreria estensibile a futuri sviluppi, viene definita l'interfaccia **Connector**, dotata di quattro metodi principali:

- **Write**: per scrivere un dato nel connettore;
- **Read**: per leggere il dato disponibile;
- **Close**: per chiudere il connettore e impedire ulteriori scritture;

- `SetReadTimeout`: per definire un tempo massimo di attesa in caso di assenza di dati in ingresso.

4.4 Worker Pool

Un *worker pool* rappresenta una strategia per gestire l'esecuzione concorrente di numerosi compiti (*task*) mediante un numero limitato di lavoratori (*worker*) riutilizzabili nel tempo. Questo approccio consente di evitare la creazione eccessiva di unità di lavoro parallele, situazione che potrebbe portare alla saturazione delle risorse di sistema.

In genere, un *worker pool* è composto da un numero fisso di *worker*, determinato dal programmatore o configurato a *runtime* tramite una variabile d'ambiente. Nel linguaggio Go, la funzione `runtime.NumCPU()` permette di ottenere il numero di core fisici e virtuali disponibili, favorendo così una gestione ottimizzata delle risorse. Questo rappresenta il punto di partenza per implementare un *worker pool* efficiente, indipendentemente dall'hardware sottostante.

La libreria **goccia** estende tale concetto introducendo uno *scaler* automatico in grado di aumentare o ridurre dinamicamente il numero di *worker* in base alla quantità di *task* presenti nella coda da cui essi estraggono il lavoro. Tale coda effettua un'operazione di *fan-out*, distribuendo i *task* in ingresso ai diversi *worker*, e funge anche da buffer intermedio. A valle dei *worker*, un componente analogo svolge la funzione di *fan-in*, ovvero la ricomposizione sequenziale dei risultati derivanti dal processamento dei *task*. Entrambi i componenti sono implementati mediante *ring buffer* MPMC.

Lo *scaler* valuta periodicamente il numero di *worker* attivi, basandosi sulla dimensione della coda di *fan-out*. Se il numero di *task* supera la soglia di riferimento (`QueueDepthPerWorker`), viene segnalato al *worker pool* di aggiungere un nuovo *worker*; altrimenti, ne viene rimosso uno. Qualora lo *scaler* rilevi ripetutamente la necessità di ridurre i *worker*, viene applicato un tempo di *backoff* esponenziale tra le riduzioni successive. In questo modo, il *worker pool* reagisce rapidamente a un improvviso incremento di carico, riducendo poi progressivamente le risorse una volta gestito il picco.

<disegno worker pool con fan-in/fan-out>

4.5 Ingress Stage

Uno *Ingress stage* rappresenta il punto di ingresso dei dati all'interno della *pipeline* di elaborazione. Costituisce la prima componente che riceve gli input provenienti dall'esterno del sistema e li introduce nel flusso di elaborazione, alimentando così l'intera catena di *stage* successivi. In sostanza, uno *Ingress stage* agisce come una

sorgente di dati, traducendo eventi, messaggi o pacchetti provenienti da differenti protocolli o interfacce in un formato gestibile dalla *pipeline* di **goccia**.

Nel contesto della libreria, gli *Ingress stage* sono progettati per essere altamente performanti, concorrenti e facilmente integrabili con i meccanismi di comunicazione già definiti dal framework. Ogni *Ingress stage* è quindi responsabile di stabilire una connessione con la sorgente, acquisire i dati in ingresso, trasformarli nel formato previsto e inoltrarli verso lo *stage* successivo.

La libreria **goccia** fornisce diversi tipi di *Ingress stage*, ciascuno progettato per scenari operativi o fonti di dati differenti.

4.5.1 Ticker

Lo *Ticker stage* rappresenta l'*Ingress stage* più semplice messo a disposizione dalla libreria **goccia** ed è concepito per generare in modo autonomo un flusso regolare di messaggi, senza dipendere da sorgenti esterne quali socket di rete, file o code di messaggistica. Dal punto di vista concettuale, si tratta di una sorgente periodica di eventi, particolarmente utile per casi d'uso quali il testing della *pipeline*, il benchmarking di prestazioni, l'iniezione di traffico sintetico o la generazione di "heartbeat" applicativi a intervalli costanti. Ogni "tick" corrisponde alla produzione di un messaggio che attraversa l'intera *pipeline*, permettendo di osservare il comportamento dei vari *stage* a valle in presenza di un carico controllato.

La logica di generazione dei messaggi sfrutta il `time.Ticker` della libreria standard del Go, il quale mette a disposizione un channel notificato ogni volta che l'intervallo T viene raggiunto, dove T è definito dalla configurazione dello *stage*.

```
1 for {
2     // ...
3     select {
4         case <-ctx.Done():
5             return
6         case <-ts.ticker.C:
7             // generate the ticker message
8     }
9 }
```

Listing 4.1: Ciclo principale del Ticker stage (ingress/ticker.go, metodo run)

Nel frammento di codice riportato, è possibile osservare come il costrutto `select` permetta di ascoltare su molteplici channel in modalità multiplex. In questo caso specifico, il *runtime* rimane in attesa di essere notificato dal ticker oppure della ricezione di un segnale di cancellazione del contesto (`ctx.Done()`). Quest'ultimo meccanismo è fondamentale per implementare strategie intelligenti di rilascio delle risorse, ad esempio quando il processo riceve un segnale SIGINT o

SIGTERM, consentendo così l'implementazione di uno *graceful shutdown* coerente nelle applicazioni. Per questa ragione, nelle librerie Go moderne è prassi richiedere un `context.Context` come primo argomento di funzioni che impiegano socket o richiedono tempistiche significative per la terminazione.

Il tipo di messaggio prodotto dallo *Ticker stage* è descritto dalla struct `TickerMessage` e contiene un unico campo specifico, `TickNumber`, utilizzato per numerare progressivamente i tick. Al momento della generazione del messaggio vengono popolati i campi generici di metadato (tempo di ricezione, timestamp) e viene inizializzata la *root span* per la tracciatura degli attraversamenti degli *stage* successivi, conformemente allo standard OpenTelemetry[1].

4.5.2 UDP

Lo *UDP stage* rappresenta un *Ingress stage* progettato per ricevere datagrammi *UDP* provenienti dalla rete. A differenza dello *Ticker stage*, che genera autonomamente messaggi a intervalli regolari, lo *UDP stage* rimane in ascolto su un socket di rete e resta in attesa di nuovi datagrammi in arrivo. Questa tipologia di *Ingress stage* risulta particolarmente utile in contesti dove è necessario elaborare flussi di dati provenienti da sorgenti esterne, quali sensori IoT, applicazioni remote, strumenti di monitoraggio di rete o qualsiasi sistema che comunichi tramite il protocollo *UDP*. La natura *connectionless* del protocollo *UDP* consente di ricevere messaggi da molteplici mittenti senza la necessità di stabilire connessioni esplicite, rendendolo ideale per scenari ad alto *throughput* dove la perdita occasionale di datagrammi è accettabile in cambio della bassa latenza.

La libreria **goccia** mette a disposizione tre parametri di configurazione per lo *UDP stage*: indirizzo IP, porta e dimensione del buffer. I primi due servono a impostare la sorgente da cui ricevere i pacchetti e dispongono di valori di default — "0.0.0.0" per l'indirizzo (che indica l'ascolto su tutte le interfacce di rete) e 20.000 per la porta. La dimensione del buffer utilizzato per leggere il payload dei datagrammi *UDP* è impostata di default a 1474 byte, poiché la dimensione massima per un payload Ethernet standard è di 1500 byte, da cui si sottraggono 28 byte relativi all'header *UDP*.

Internamente, lo *stage* utilizza un puntatore a una connessione *UDP* fornita dalla libreria standard del Go[2]. Tale connessione viene inizializzata nel metodo `Init` del ciclo di vita dello *stage*. Una volta che l'inizializzazione ha esito positivo, è possibile procedere con l'esecuzione dello *stage* tramite il metodo `Run`. Questo metodo implementa un ciclo `for` in cui viene richiamato il metodo `Read` (operazione bloccante) della connessione *UDP*, il quale scrive i byte ricevuti in un buffer pre-allocato alla dimensione configurata. Parallelamente al ciclo principale, viene avviata una goroutine ausiliaria il cui compito è chiudere la connessione *UDP* nel momento in cui viene ricevuta una notifica di cancellazione tramite

`context.Context`. Una volta che la connessione è chiusa, il metodo `Read` ritorna un errore di tipo `net.ErrClosed`[3], permettendo così al ciclo di uscire in modo ordinato.

```
1 go func() {
2     <-ctx.Done()
3     us.conn.Close()
4 }()
5
6 buf := make([]byte, us.bufferSize)
7
8 for {
9     // Read the UDP payload
10    n, err := us.conn.Read(buf)
11    if err != nil {
12        // Check if the connection is closed
13        if errors.Is(err, net.ErrClosed) {
14            // Check if caused by context cancellation
15            select {
16                case <-ctx.Done():
17                    return
18                default:
19            }
20        }
21
22        // ...
23    }
24
25    // Handle the buffer and send the message ...
26 }
```

Listing 4.2: Ciclo principale dello UDP stage (ingress/udp.go, metodo `run`)

Il tipo di messaggio prodotto dallo *UDP stage* è descritto dalla struct `UDPMessage`, contenente due campi specifici: `Payload` e `PayloadSize`. Il messaggio implementa l'interfaccia `message.Serializable` definita nel package `message`, consentendo il collegamento di differenti tipi di *Processor stage* a valle che accettano messaggi recanti buffer di byte. Al fine di soddisfare l'interfaccia, è sufficiente implementare il metodo `GetBytes`, il quale restituisce una slice di byte corrispondente al campo `Payload` del messaggio.

```
1 func (um *UDPMessage) GetBytes() []byte {
2     return um.Payload
3 }
```

Listing 4.3: Metodo `GetBytes` di `UDPMessage` (ingress/udp.go)

Poiché il payload è caratterizzato da dimensioni significative, al fine di ridurre il carico sul *garbage collector* attraverso il riuso della memoria e l'eliminazione di continue allocazioni e deallocazioni di migliaia di oggetti, si è optato per l'utilizzo di un *object pool* (`sync.Pool`)[4]. Gli oggetti `UDPMessage` vengono quindi pre-allocati nel pool e riutilizzati per ogni nuovo datagramma ricevuto, riducendo così la pressione sul sistema di gestione della memoria durante l'elaborazione ad alto throughput.

Lo *stage* espone due metriche destinate al monitoraggio del numero di messaggi e byte ricevuti, implementate tramite l'uso di contatori asincroni[5]. Tali metriche possono essere impiegate per monitorare il *throughput* in ingresso alla *pipeline*, fornendo visibilità sulla velocità di ricezione dei dati dalla sorgente di rete.

4.5.3 TCP

Lo *TCP stage* rappresenta un *Ingress stage* progettato per ricevere flussi dati da connessioni *TCP*. A differenza dello *UDP stage*, che gestisce singoli datagrammi provenienti da molteplici mittenti senza stato, il *TCP stage* stabilisce connessioni persistenti con i client e mantiene lo stato della comunicazione per ciascuna connessione. Questa caratteristica lo rende particolarmente adatto per scenari in cui l'affidabilità del trasporto è critica, quali l'acquisizione di log da sistemi remoti, la ricezione di comandi da applicazioni client o l'integrazione con protocolli applicativi che richiedono una comunicazione orientata allo *stream*. Diversamente dallo *Ticker stage* e dallo *UDP stage*, il *TCP stage* introduce una complessità significativamente maggiore, poiché deve gestire molteplici connessioni concorrenti, ciascuna potenzialmente caratterizzata da uno stato diverso, e deve risolvere il problema fondamentale di come delimitare i messaggi all'interno di un flusso di byte continuo.

La libreria **goccia** mette a disposizione una configurazione flessibile per il *TCP stage*, incapsulata nella struct `TCPConfig`. I parametri fondamentali sono analoghi a quelli dello *UDP stage*: un indirizzo IP e una porta definiscono il punto di ascolto su cui il server *TCP* rimane in attesa di connessioni in ingresso, mentre un buffer di lettura viene utilizzato per acquisire i dati dalla connessione. Tuttavia, il *TCP stage* introduce ulteriori parametri specifici per la gestione avanzata del flusso.

Il parametro `ReadTimeout` specifica il tempo massimo di inattività consentito su una connessione prima che essa venga forzatamente chiusa. Questo meccanismo protegge il server da client che si collegano e rimangono silenti indefinitamente, occupando risorse preziose del sistema. Il parametro `MaxMessageSize` (default 4 MB) definisce la dimensione massima consentita per un messaggio; qualora il buffer accumulato superi questo valore, la connessione viene chiusa al fine di prevenire attacchi di *denial of service* basati su messaggi abnormemente grandi.

Un aspetto cruciale della configurazione è il *framing mode*, definito dal campo `FramingMode`, il quale specifica come i messaggi vengono delimitati all'interno del flusso *TCP*. Sono supportate due modalità distinte:

- ***Delimited*** (default): I messaggi sono separati da un delimitatore, tipicamente una sequenza di byte come `"\r\n"`, impostabile dall'utente nel campo `Delimiter`. In questa modalità, il *parser* ricerca la sequenza delimitatrice all'interno del buffer accumulato e la utilizza come marcatore di fine messaggio. Esempi di protocolli di livello superiore che impiegano questa strategia sono HTTP/1.x, SMTP e FTP.
- ***Length-Prefixed***: I messaggi sono preceduti da un header contenente la lunghezza del messaggio. Questa modalità introduce parametri aggiuntivi per estrarre la lunghezza del payload a partire da un header:
 - `HeaderLen` (default 16 byte): la dimensione totale dell'header.
 - `MessageLengthFieldOffset` (default 0): l'offset all'interno dell'header dove inizia il campo di lunghezza.
 - `MessageLengthFieldLen`: la dimensione del campo di lunghezza (1, 2, 4 o 8 byte).
 - `MessageLengthFieldEndianness`: l'ordine dei byte (*little-endian* o *big-endian*) del campo di lunghezza.

Infine, il parametro `OutputQueueSize` specifica la dimensione della coda interna (*fan-in*) che media tra le goroutine dedicate alle connessioni e il *connector* verso lo *stage* successivo.

Il *TCP stage* implementa un'architettura basata su molteplici goroutine sincronizzate. Il ciclo principale si occupa di accettare nuove connessioni tramite il metodo `Accept` del *listener TCP*[6] e, per ciascuna nuova connessione in ingresso, avvia una nuova goroutine dedicata alla gestione della relativa comunicazione. In questo modo, si ottiene un pattern di *fan-out*, in cui una singola goroutine principale genera *N* goroutine worker, una per ogni connessione cliente.

Questa architettura è resa possibile dal *runtime* di Go, il quale è in grado di gestire un numero elevato di goroutine e di schedularle efficientemente su un numero finito di thread del sistema operativo.

```
1 for {
2     // ...
3
4     conn, err := ts.listener.Accept()
5     // Check and handle the error ...
6
7     // Spawn a goroutine to handle the connection
```

```
8  go ts.handleConn(ctx, conn)
9  }
```

Listing 4.4: Ciclo principale del TCP stage (ingress/tcp.go, metodo run)

Per quanto riguarda la gestione della singola connessione, il flusso è concettualmente simile a quello dello *UDP stage*, con l'eccezione rilevante della divisione del *stream* in messaggi. Tale divisione richiede l'utilizzo di un buffer ausiliario di accumulazione, oltre al buffer di lettura. Il buffer di accumulazione è fondamentale nei casi in cui un messaggio risulti frammentato su più letture: ad esempio, se il buffer di lettura è di 4 KB e il messaggio trasmesso sulla connessione è di 8 KB, saranno necessarie due letture i cui contenuti dovranno essere accumulati sequenzialmente.

```
1  // Preallocate the accumulator
2  accBaseCap := min(4*ts.bufferSize, ts.maxMsgSize)
3  acc := make([]byte, 0, accBaseCap)
4
5  // ...
6
7  for {
8      // ...
9
10     // Set the read deadline
11     conn.SetReadDeadline(time.Now().Add(ts.readTimeout))
12
13     // Read the TCP stream
14     n, err := conn.Read(buf)
15     // Check and handle the error ...
16
17     // Append the new bytes to the accumulator
18     acc = append(acc, buf[:n]...)
19
20     // Prevent accumulator from growing too large
21     if len(acc) > ts.maxMsgSize {
22         ts.tel.LogWarn("message too large, closing connection")
23         return
24     }
25
26     for {
27         // Divide the accumulator into messages ...
28     }
29
30     // ...
31 }
```

Listing 4.5: Buffer per la gestione connessione del TCP stage (ingress/tcp.go, metodo handleConn)

Dopo l'accumulazione dei byte trasmessi, entra in gioco la logica di divisione dei messaggi secondo la modalità configurata nello *stage* (*delimited* oppure *length-prefixed*). In modalità delimitata, la ricerca della sequenza delimitatrice è effettuata tramite `bytes.Index`; in modalità *length-prefixed*, il *parser* estrae la lunghezza dal campo di header designato, utilizzando funzioni che gestiscono le diverse combinazioni di lunghezza e *endianess* supportate.

```
1 // ...
2
3 for {
4     accLen := len(acc)
5
6     // If the accumulator is smaller than the minimum length,
7     // continue reading the TCP stream
8     if accLen < minAccLen {
9         continue loop
10    }
11
12    // Get the length of the message.
13    msgLen := 0
14    totLen := 0
15    switch ts.framingMode {
16    case TCPFramingModeDelimited:
17        // Search for the delimiter
18        msgLen = bytes.Index(acc, ts.delimiter)
19        totLen = msgLen + ts.delimiterLen
20
21    case TCPFramingModeLengthPrefixed:
22        msgLen = ts.parseHeader(acc[:ts.headerLen])
23        totLen = msgLen + ts.headerLen
24    }
25
26    if msgLen == -1 || accLen < totLen {
27        // If the message length is not found or the
28        // accumulator is too small,
29        // break the loop and continue reading the TCP stream
30        break
31    }
32
33    // Extract the message
34    msg := acc[:totLen]
```

```
34
35 // Handle the message and send the result to the output
    connector ...
36
37 // Remove the message from the accumulator
38 acc = acc[totLen:]
39
40 // Check if the accumulator should be reset ...
41 }
42
43 // ...
```

Listing 4.6: Divisione messaggi del TCP stage (ingress/tcp.go, metodo handleConn)

Il tipo di messaggio prodotto dal *TCP stage* è descritto dalla struct `TCPMessage`, la quale implementa l'interfaccia di serializzazione, in modo analogo al messaggio dello *UDP stage*. A differenza di `UDPMessage`, `TCPMessage` non utilizza un *object pool*, poiché i messaggi *TCP* possono avere dimensioni molto variabili e l'impiego di un pool con buffer pre-allocato potrebbe risultare inefficiente o insufficiente. Il messaggio porta con sé il payload in byte (`Message`), la dimensione del payload (`MessageSize`) e l'indirizzo remoto della connessione (`RemoteAddr`), fornendo così sia il contenuto informativo sia il contesto di provenienza.

Dal punto di vista dell'osservabilità, il *TCP stage* è per molti aspetti simile allo *UDP stage*. Anche in questo caso sono presenti metriche per il numero totale di messaggi e di byte ricevuti, che consentono di monitorare il *throughput* complessivo della *pipeline*. In aggiunta, viene introdotta una metrica specifica, `open_connections`, che traccia il numero di connessioni *TCP* attualmente aperte. Tale metrica offre visibilità sul grado di utilizzo delle risorse del server e permette di individuare con facilità situazioni anomale, come un numero insolitamente elevato di connessioni persistenti o client che non rilasciano correttamente le proprie sessioni.

- Kafka
- File
- eBPF

4.6 Stage di processamento

- Cannelloni encoder/decoder
- CAN (acmelib)

- CSV encoder/decoder
- Custom
- Filter
- Tee
- ROB

4.7 Stage di egress

- UDP
- TCP
- Kafka
- File
- QuestDB
- Sink

4.8 Observabilità con OpenTelemetry

- Metriche
- Distributed tracing

Capitolo 5

Implementazione

5.1 Organizzazione codice

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Tristique senectus et netus et malesuada fames ac turpis. Pretium nibh ipsum consequat nisl vel pretium lectus quam. Urna molestie at elementum eu facilisis sed odio morbi quis. Sed egestas egestas fringilla phasellus faucibus scelerisque eleifend. At in tellus integer feugiat. Mauris rhoncus aenean vel elit scelerisque mauris pellentesque pulvinar. Commodo sed egestas egestas fringilla. Nunc lobortis mattis aliquam faucibus purus in massa. Facilisis magna etiam tempor orci eu lobortis elementum nibh. Elementum curabitur vitae nunc sed velit dignissim. Neque volutpat ac tincidunt vitae. Massa id neque aliquam vestibulum morbi blandit cursus risus at. Porta non pulvinar neque laoreet suspendisse interdum consectetur. Turpis in eu mi bibendum. Ut tristique et egestas quis ipsum suspendisse. Integer quis auctor elit sed vulputate mi sit amet. Viverra nam libero justo laoreet sit amet cursus.

5.2 Interfacce principali

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Tristique senectus et netus et malesuada fames ac turpis. Pretium nibh ipsum consequat nisl vel pretium lectus quam. Urna molestie at elementum eu facilisis sed odio morbi quis. Sed egestas egestas fringilla phasellus faucibus scelerisque eleifend. At in tellus integer feugiat. Mauris rhoncus aenean vel elit scelerisque mauris pellentesque pulvinar. Commodo sed egestas egestas fringilla. Nunc lobortis mattis aliquam faucibus purus in massa. Facilisis magna etiam tempor orci eu lobortis elementum nibh. Elementum curabitur vitae nunc sed velit dignissim. Neque volutpat ac tincidunt vitae. Massa id neque

aliquam vestibulum morbi blandit cursus risus at. Porta non pulvinar neque laoreet suspendisse interdum consectetur. Turpis in eu mi bibendum. Ut tristique et egestas quis ipsum suspendisse. Integer quis auctor elit sed vulputate mi sit amet. Viverra nam libero justo laoreet sit amet cursus.

5.3 Dettagli ring buffer

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Tristique senectus et netus et malesuada fames ac turpis. Pretium nibh ipsum consequat nisl vel pretium lectus quam. Urna molestie at elementum eu facilisis sed odio morbi quis. Sed egestas egestas fringilla phasellus faucibus scelerisque eleifend. At in tellus integer feugiat. Mauris rhoncus aenean vel elit scelerisque mauris pellentesque pulvinar. Commodo sed egestas egestas fringilla. Nunc lobortis mattis aliquam faucibus purus in massa. Facilisis magna etiam tempor orci eu lobortis elementum nibh. Elementum curabitur vitae nunc sed velit dignissim. Neque volutpat ac tincidunt vitae. Massa id neque aliquam vestibulum morbi blandit cursus risus at. Porta non pulvinar neque laoreet suspendisse interdum consectetur. Turpis in eu mi bibendum. Ut tristique et egestas quis ipsum suspendisse. Integer quis auctor elit sed vulputate mi sit amet. Viverra nam libero justo laoreet sit amet cursus.

5.4 Dettagli worker pool

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Tristique senectus et netus et malesuada fames ac turpis. Pretium nibh ipsum consequat nisl vel pretium lectus quam. Urna molestie at elementum eu facilisis sed odio morbi quis. Sed egestas egestas fringilla phasellus faucibus scelerisque eleifend. At in tellus integer feugiat. Mauris rhoncus aenean vel elit scelerisque mauris pellentesque pulvinar. Commodo sed egestas egestas fringilla. Nunc lobortis mattis aliquam faucibus purus in massa. Facilisis magna etiam tempor orci eu lobortis elementum nibh. Elementum curabitur vitae nunc sed velit dignissim. Neque volutpat ac tincidunt vitae. Massa id neque aliquam vestibulum morbi blandit cursus risus at. Porta non pulvinar neque laoreet suspendisse interdum consectetur. Turpis in eu mi bibendum. Ut tristique et egestas quis ipsum suspendisse. Integer quis auctor elit sed vulputate mi sit amet. Viverra nam libero justo laoreet sit amet cursus.

5.5 Dettagli ROB e stimatore EMA

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Tristique senectus et netus et malesuada fames ac turpis. Pretium nibh ipsum consequat nisl vel pretium lectus quam. Urna molestie at elementum eu facilisis sed odio morbi quis. Sed egestas egestas fringilla phasellus faucibus scelerisque eleifend. At in tellus integer feugiat. Mauris rhoncus aenean vel elit scelerisque mauris pellentesque pulvinar. Commodore sed egestas egestas fringilla. Nunc lobortis mattis aliquam faucibus purus in massa. Facilisis magna etiam tempor orci eu lobortis elementum nibh. Elementum curabitur vitae nunc sed velit dignissim. Neque volutpat ac tincidunt vitae. Massa id neque aliquam vestibulum morbi blandit cursus risus at. Porta non pulvinar neque laoreet suspendisse interdum consectetur. Turpis in eu mi bibendum. Ut tristique et egestas quis ipsum suspendisse. Integer quis auctor elit sed vulputate mi sit amet. Viverra nam libero justo laoreet sit amet cursus.

Capitolo 6

Benchmark e valutazione prestazioni

6.1 Setup sperimentale

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Tristique senectus et netus et malesuada fames ac turpis. Pretium nibh ipsum consequat nisl vel pretium lectus quam. Urna molestie at elementum eu facilisis sed odio morbi quis. Sed egestas egestas fringilla phasellus faucibus scelerisque eleifend. At in tellus integer feugiat. Mauris rhoncus aenean vel elit scelerisque mauris pellentesque pulvinar. Commodo sed egestas egestas fringilla. Nunc lobortis mattis aliquam faucibus purus in massa. Facilisis magna etiam tempor orci eu lobortis elementum nibh. Elementum curabitur vitae nunc sed velit dignissim. Neque volutpat ac tincidunt vitae. Massa id neque aliquam vestibulum morbi blandit cursus risus at. Porta non pulvinar neque laoreet suspendisse interdum consectetur. Turpis in eu mi bibendum. Ut tristique et egestas quis ipsum suspendisse. Integer quis auctor elit sed vulputate mi sit amet. Viverra nam libero justo laoreet sit amet cursus.

6.2 Confronto con altri framework

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Tristique senectus et netus et malesuada fames ac turpis. Pretium nibh ipsum consequat nisl vel pretium lectus quam. Urna molestie at elementum eu facilisis sed odio morbi quis. Sed egestas egestas fringilla phasellus faucibus scelerisque eleifend. At in tellus integer feugiat. Mauris rhoncus aenean vel elit scelerisque mauris pellentesque pulvinar. Commodo sed egestas

egestas fringilla. Nunc lobortis mattis aliquam faucibus purus in massa. Facilisis magna etiam tempor orci eu lobortis elementum nibh. Elementum curabitur vitae nunc sed velit dignissim. Neque volutpat ac tincidunt vitae. Massa id neque aliquam vestibulum morbi blandit cursus risus at. Porta non pulvinar neque laoreet suspendisse interdum consectetur. Turpis in eu mi bibendum. Ut tristique et egestas quis ipsum suspendisse. Integer quis auctor elit sed vulputate mi sit amet. Viverra nam libero justo laoreet sit amet cursus.

6.3 Analisi vantaggi e limiti

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Tristique senectus et netus et malesuada fames ac turpis. Pretium nibh ipsum consequat nisl vel pretium lectus quam. Urna molestie at elementum eu facilisis sed odio morbi quis. Sed egestas egestas fringilla phasellus faucibus scelerisque eleifend. At in tellus integer feugiat. Mauris rhoncus aenean vel elit scelerisque mauris pellentesque pulvinar. Commodo sed egestas egestas fringilla. Nunc lobortis mattis aliquam faucibus purus in massa. Facilisis magna etiam tempor orci eu lobortis elementum nibh. Elementum curabitur vitae nunc sed velit dignissim. Neque volutpat ac tincidunt vitae. Massa id neque aliquam vestibulum morbi blandit cursus risus at. Porta non pulvinar neque laoreet suspendisse interdum consectetur. Turpis in eu mi bibendum. Ut tristique et egestas quis ipsum suspendisse. Integer quis auctor elit sed vulputate mi sit amet. Viverra nam libero justo laoreet sit amet cursus.

6.4 Impatto OpenTelemetry

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Tristique senectus et netus et malesuada fames ac turpis. Pretium nibh ipsum consequat nisl vel pretium lectus quam. Urna molestie at elementum eu facilisis sed odio morbi quis. Sed egestas egestas fringilla phasellus faucibus scelerisque eleifend. At in tellus integer feugiat. Mauris rhoncus aenean vel elit scelerisque mauris pellentesque pulvinar. Commodo sed egestas egestas fringilla. Nunc lobortis mattis aliquam faucibus purus in massa. Facilisis magna etiam tempor orci eu lobortis elementum nibh. Elementum curabitur vitae nunc sed velit dignissim. Neque volutpat ac tincidunt vitae. Massa id neque aliquam vestibulum morbi blandit cursus risus at. Porta non pulvinar neque laoreet suspendisse interdum consectetur. Turpis in eu mi bibendum. Ut tristique et egestas quis ipsum suspendisse. Integer quis auctor elit sed vulputate mi sit amet. Viverra nam libero justo laoreet sit amet cursus.

Capitolo 7

Applicazioni e sviluppi futuri

7.1 Telemetria in SquadraCorse

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Tristique senectus et netus et malesuada fames ac turpis. Pretium nibh ipsum consequat nisl vel pretium lectus quam. Urna molestie at elementum eu facilisis sed odio morbi quis. Sed egestas egestas fringilla phasellus faucibus scelerisque eleifend. At in tellus integer feugiat. Mauris rhoncus aenean vel elit scelerisque mauris pellentesque pulvinar. Commodo sed egestas egestas fringilla. Nunc lobortis mattis aliquam faucibus purus in massa. Facilisis magna etiam tempor orci eu lobortis elementum nibh. Elementum curabitur vitae nunc sed velit dignissim. Neque volutpat ac tincidunt vitae. Massa id neque aliquam vestibulum morbi blandit cursus risus at. Porta non pulvinar neque laoreet suspendisse interdum consectetur. Turpis in eu mi bibendum. Ut tristique et egestas quis ipsum suspendisse. Integer quis auctor elit sed vulputate mi sit amet. Viverra nam libero justo laoreet sit amet cursus.

7.2 Miglioramenti futuri

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Tristique senectus et netus et malesuada fames ac turpis. Pretium nibh ipsum consequat nisl vel pretium lectus quam. Urna molestie at elementum eu facilisis sed odio morbi quis. Sed egestas egestas fringilla phasellus faucibus scelerisque eleifend. At in tellus integer feugiat. Mauris rhoncus aenean vel elit scelerisque mauris pellentesque pulvinar. Commodo sed egestas egestas fringilla. Nunc lobortis mattis aliquam faucibus purus in massa. Facilisis magna etiam tempor orci eu lobortis elementum nibh. Elementum curabitur vitae nunc sed velit dignissim. Neque volutpat ac tincidunt vitae. Massa id neque

aliquam vestibulum morbi blandit cursus risus at. Porta non pulvinar neque laoreet suspendisse interdum consectetur. Turpis in eu mi bibendum. Ut tristique et egestas quis ipsum suspendisse. Integer quis auctor elit sed vulputate mi sit amet. Viverra nam libero justo laoreet sit amet cursus.

Capitolo 8

Conclusioni

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Porttitor eget dolor morbi non arcu risus quis varius. Libero id faucibus nisl tincidunt. Neque laoreet suspendisse interdum consectetur libero id faucibus nisl tincidunt. Scelerisque in dictum non consectetur a erat. Leo a diam sollicitudin tempor id eu. Sodales ut eu sem integer vitae justo eget magna fermentum. A cras semper auctor neque vitae. Cursus euismod quis viverra nibh cras pulvinar. Mi tempus imperdiet nulla malesuada pellentesque elit eget gravida cum. Dictum at tempor commodo ullamcorper a lacus vestibulum sed. Ultricies mi eget mauris pharetra. In pellentesque massa placerat dui ultricies lacus. Fringilla phasellus faucibus scelerisque eleifend donec pretium vulputate.

Aliquam sem fringilla ut morbi tincidunt augue interdum. Risus at ultrices mi tempus imperdiet nulla malesuada pellentesque. Semper quis lectus nulla at volutpat. Nullam non nisi est sit amet facilisis. Eget velit aliquet sagittis id consectetur purus ut faucibus pulvinar. Ultricies tristique nulla aliquet enim tortor at auctor urna nunc. Pharetra diam sit amet nisl suscipit adipiscing bibendum est ultricies. Amet facilisis magna etiam tempor. Nunc non blandit massa enim nec dui nunc mattis. At ultrices mi tempus imperdiet nulla malesuada pellentesque. Cursus metus aliquam eleifend mi in nulla posuere. In eu mi bibendum neque egestas congue quisque. Augue eget arcu dictum varius dui at consectetur.

Bibliografia

- [1] Cloud Native Computing Foundation. *OpenTelemetry Trace API Specification*. 2024. URL: <https://opentelemetry.io/docs/specs/otel/trace/api/> (visitato il giorno 29/12/2025) (cit. a p. 11).
- [2] The Go Authors. *Package net - UDPCConn Type*. 2025. URL: <https://pkg.go.dev/net#UDPCConn> (visitato il giorno 29/12/2025) (cit. a p. 11).
- [3] The Go Authors. *Package net - ErrClosed Variable*. 2025. URL: <https://pkg.go.dev/net#ErrClosed> (visitato il giorno 29/12/2025) (cit. a p. 12).
- [4] The Go Authors. *Package sync - Pool Type*. 2025. URL: <https://pkg.go.dev/sync#Pool> (visitato il giorno 29/12/2025) (cit. a p. 13).
- [5] Cloud Native Computing Foundation. *OpenTelemetry Metrics API Specification - Asynchronous Counter*. 2025. URL: <https://opentelemetry.io/docs/specs/otel/metrics/api/#asynchronous-counter> (visitato il giorno 29/12/2025) (cit. a p. 13).
- [6] The Go Authors. *Package net - TCPLListener.Accept Method*. 2025. URL: <https://pkg.go.dev/net#TCPLListener.Accept> (visitato il giorno 30/12/2025) (cit. a p. 14).